

Heaven's light is our guide

Rajshahi University of Engineering & Technology (RUET)

Rajshahi-6204, Bangladesh

Department of Computer Science & Engineering

CSE 3206: Software Engineering Sessional

Lab 3: Design Pattern Analysis, Implementation and Code Review

LAB LECTURE DOCUMENTATION

Iterator & Mediator Design Patterns

CampusConnect: a Facebook-like feed and a Messenger-like chat, implemented in C++17

Presentation Group 7 — Assigned Patterns: Iterator & Mediator

Name	Student ID	GitHub Username
Abdullah Al Kafi (Group Leader)	2203139	@Kafi2611
Sadaf Rahman	2203140	@sadaf532
Sakila Akter	2203141	@SakilaAkter

GitHub Repository: <https://github.com/Kafi2611/CSE3206-Group7-Iterator-Mediator>

Submitted to: Course Teacher, CSE 3206 — Department of CSE, RUET

Contents

1. Project Overview.....	3
1.1 About CampusConnect.....	3
1.2 Problem Analysis and Pattern Selection.....	3
1.3 Technology and Quality Summary.....	3
1.4 Team and Responsibilities.....	4
Part A — Iterator Pattern (CampusConnect Feed).....	5
A.1 Pattern Name.....	5
A.2 Category.....	5
A.3 Intent.....	5
A.4 Problem Statement.....	5
A.5 Motivation – why the normal implementation is poor.....	6
A.6 Pattern Structure (UML Class Diagram).....	7
A.7 Class Responsibilities.....	9
A.8 Code Implementation (step by step).....	9
A.9 Execution Flow (Object Interaction).....	15
A.10 Advantages.....	15
A.11 Limitations.....	16
A.12 Real-life Applications.....	16
A.13 Industry Examples.....	16
A.14 Demonstration (Live Execution).....	16
A.15 Conclusion.....	17
Part B — Mediator Pattern (CampusConnect Messenger).....	18
B.1 Pattern Name.....	18
B.2 Category.....	18
B.3 Intent.....	18
B.4 Problem Statement.....	18
B.5 Motivation – why the normal implementation is poor.....	19
B.6 Pattern Structure (UML Class Diagram).....	20
B.7 Class Responsibilities.....	21
B.8 Code Implementation (step by step).....	22
B.9 Execution Flow (Object Interaction).....	26
B.10 Advantages.....	27
B.11 Limitations.....	27
B.12 Real-life Applications.....	27
B.13 Industry Examples.....	27
B.14 Demonstration (Live Execution).....	28
B.15 Conclusion.....	29
Part C — Comparing Iterator and Mediator.....	30
C.1 Side-by-side Comparison.....	30
C.2 How the Two Patterns Work Together.....	30
C.3 Related Patterns.....	30
Part D — Testing and Code Quality.....	31
Appendices.....	32
Appendix A – How to Build and Run.....	32
Appendix B – Repository Structure.....	32
Appendix C – References.....	32

1. Project Overview

1.1 About CampusConnect

To demonstrate our two assigned **behavioral** design patterns in one realistic story, we built **CampusConnect**, a small console-based social platform for RUET students written in **C++17**. It has two independent modules:

Module	Inspired by	Pattern	What the user can do
CampusConnect Feed	Facebook	Iterator	see friends, <i>People You May Know</i> and a news feed
CampusConnect Messenger	Messenger	Mediator	send direct and group messages, block users, receive offline messages, chat with a help-desk bot

The names *Facebook* and *Messenger* are used only to describe the kind of application; the project uses its own name and contains no third-party branding.

1.2 Problem Analysis and Pattern Selection

Before coding we analysed the core design problem of each module and compared candidate patterns. The table summarises why Iterator and Mediator are the most appropriate choices.

	Feed module	Messenger module
Core problem	Many traversals (friends, suggestions, feed) over different internal structures (map, graph, vector) that the UI must not depend on.	Many objects (users, bots) that must communicate many-to-many under shared rules (groups, blocking, offline queue).
Alternatives considered	Public getters for the containers; Visitor ; Composite .	Direct references between users; Observer (publish/subscribe); Facade .
Why they fall short	Getters break encapsulation and push traversal code into the UI. Visitor adds <i>operations</i> over elements but does not provide <i>sequential access</i> or multiple cursors. Composite models tree structure, not traversal order.	Direct references create $n(n-1)/2$ couplings and scatter the rules. Observer gives one-to-many notifications, but the rules still live in many subscribers. Facade simplifies access to a subsystem, but its classes do not communicate through it.
Chosen pattern	Iterator – sequential access through one interface, one class per traversal algorithm, independent cursors, representation hidden.	Mediator – colleagues know only the mediator; every interaction rule is centralised in one replaceable object.

1.3 Technology and Quality Summary

Item	Details
Language / standard	C++17, no third-party libraries
Build	CMake 3.14+ (any IDE), or one-click <code>build.bat</code> (Windows) / <code>build.sh</code> (Linux, macOS, Git Bash)
Compilers verified	GCC 13 and Clang 18 – 0 warnings with <code>-Wall -Wextra -Wpedantic -Wshadow -Wconversion</code>
Unit tests	33 tests (16 Iterator + 17 Mediator) with a tiny built-in framework – all passing; $\approx 99\%$ line coverage
Static analysis	clang-tidy (bugprone, modernize, performance, readability, cppcoreguidelines, misc)

Item	Details
Memory safety	AddressSanitizer + UndefinedBehaviorSanitizer and Valgrind – 0 errors, 0 leaks
Continuous integration	GitHub Actions builds and runs all tests on every push and pull request
Size	≈ 1,400 lines of C++ in the two modules, ≈ 520 lines of tests

1.4 Team and Responsibilities

Member	ID	Main responsibilities
Abdullah Al Kafi (Leader)	2203139	Project architecture; Iterator module (CampusConnect Feed); build system and CI; GitHub repository owner; presentation lead and live demo
Sadaf Rahman	2203140	Mediator module (CampusConnect Messenger); Mediator UML diagrams; presents Mediator structure and code walkthrough
Sakila Akter	2203141	Unit tests; lab documentation and code review report; presents advantages, applications, comparison and code quality

All source code, diagrams and documents are in the GitHub repository: <https://github.com/Kafi2611/CSE3206-Group7-Iterator-Mediator>

Part A — Iterator Pattern (CampusConnect Feed)

Scenario in one line

CampusConnect is a **Facebook-like** campus social network. Its screen must list a user's **friends**, show **People You May Know** and render a **News Feed**, while the data lives in a map, a friendship graph and a vector. The Iterator pattern lets the screen walk all of them with the same three calls: `hasNext()`, `next()`, `reset()`.

A.1 Pattern Name

Iterator Pattern (also known as **Cursor**).

In one sentence: *walk through the elements of a collection one at a time without knowing, or depending on, how the collection stores them.*

A.2 Category

Creational	Structural	Behavioral
—	—	✓ Iterator belongs here

Iterator is an **object-behavioral** pattern: it is about how responsibilities are divided between objects. The *collection* stores data, the *iterator* knows how to traverse it, and the *client* only consumes elements.

A.3 Intent

Intent (Gang of Four, 1994)

“Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.”

Why the pattern was created:

- A collection should not have to publish its internal containers (arrays, lists, trees, graphs) just so that others can loop over it.
- The **traversal algorithm** (order, filtering, depth-first vs breadth-first, pagination) is a separate responsibility from **storing** data and deserves its own class.
- Clients should use **one uniform interface** for every collection, so the same loop works everywhere.
- Several traversals must be able to run **at the same time** over the same collection, each with its own position.

A.4 Problem Statement

CampusConnect stores its data in three different structures, chosen for efficiency:

Data	Internal structure	Why this structure
Profiles	<code>std::map<std::string, Profile></code>	fast lookup by id, sorted ids
Friendships	adjacency list <code>std::map<std::string, std::set<std::string>></code> (an undirected graph)	mutual friendship, no duplicates, $O(\log n)$ membership test
Posts	<code>std::vector<Post></code> in publishing order	cheap append, cache friendly

The user interface (**FeedScreen**) needs **three different traversals** over this data:

1. **Friends list** – the direct friends of a user (graph neighbours).
2. **People You May Know** – friends-of-friends who are not yet friends, ranked by the number of mutual friends (a breadth-first search of depth 2).
3. **News Feed** – posts written by the user and their friends only, newest first (filter + sort).

Further requirements: new traversals (e.g. *Top posts*, *Birthdays today*) must be addable without editing the UI; two traversals must be able to run at the same time (finding mutual friends needs two cursors); and if a new post arrives while the user is scrolling, the program must **detect** it instead of reading invalid memory.

The design problem

How can the UI walk three differently structured collections in several different orders **without knowing how any of them are stored**, and without rewriting the UI every time a new ordering is needed?

A.5 Motivation – why the normal implementation is poor

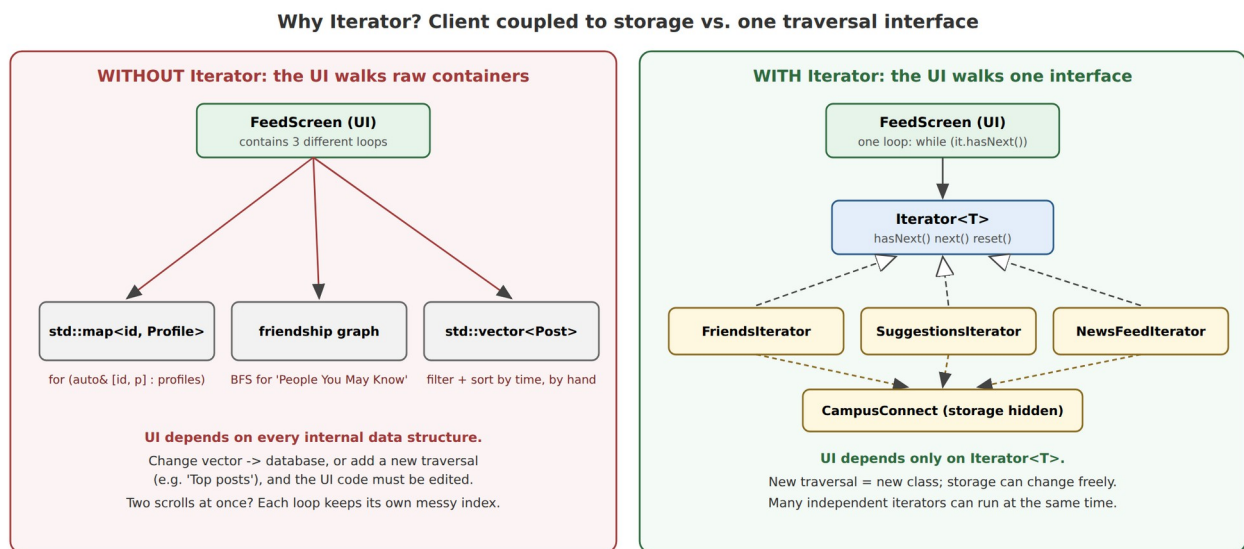


Figure 1: Without Iterator the UI depends on every data structure; with Iterator it depends on a single interface.

The obvious, “normal” solution is to let the screen reach into the network's containers and write the loops itself:

Naive approach (anti-example, not part of the project)

```
// NAIVE design (what we did NOT do): the UI knows every internal container
void FeedScreen::showFeed(const Network& net, const std::string& me) {
    std::set<std::string> circle = net.friends.at(me); // reaches into the graph
    circle.insert(me);
    std::vector<Post> feed;
    for (const Post& p : net.posts) // reaches into the vector
        if (circle.count(p.authorId)) feed.push_back(p);
    std::sort(feed.begin(), feed.end(), [](const Post& a, const Post& b) {
        return a.timestamp > b.timestamp; // traversal policy in the UI
    });
    for (const Post& p : feed) print(p);
}
// ...plus a completely different loop (with a BFS!) for "People You May Know",
// and another one for the friends list.
```

Why this is poor:

- **Broken encapsulation** – **friends** and **posts** must be public, so any class can corrupt them.

- **Single Responsibility violated** – the screen mixes rendering with filtering, sorting and graph search.
- **Open/Closed violated** – every new ordering (*Top posts*) means editing the UI class again.
- **Fragile** – replacing the vector by a database, a linked list or a paginated web API breaks every loop in every screen.
- **Duplication** – each screen (web, mobile, notifications) re-implements the same loops.
- **Unsafe** – references into a `std::vector` become dangling after a reallocation; nothing detects that the data changed while scrolling.

A.6 Pattern Structure (UML Class Diagram)

The full UML class diagram is shown on the next page in landscape orientation (**Figure 2**). It maps onto the classic Gang-of-Four participants as follows:

GoF participant	Our class(es)	File
Iterator	<code>Iterator<T></code> (aliases <code>ProfileIterator</code> , <code>PostIterator</code>)	<code>iterator/include/Iterator.hpp</code>
ConcreteIterator	<code>FriendsIterator</code> , <code>SuggestionsIterator</code> , <code>NewsFeedIterator</code> (share <code>FailFastIterator<T, Key></code>)	<code>iterator/include/FeedIterators.hpp</code> , <code>src/FeedIterators.cpp</code>
Aggregate	<code>SocialNetwork</code>	<code>iterator/include/SocialNetwork.hpp</code>
ConcreteAggregate	<code>CampusConnect</code>	<code>iterator/include/CampusConnect.hpp</code> , <code>src/CampusConnect.cpp</code>
Client	<code>FeedScreen</code> , <code>findMutualFriends()</code> , <code>main()</code>	<code>iterator/include/FeedScreen.hpp</code> , <code>src/FeedScreen.cpp</code> , <code>main.cpp</code>

UML legend: dashed line + hollow triangle = implements an interface; solid line + hollow triangle = inherits; filled diamond = composition (owns); dashed open arrow = dependency / «creates»; solid open arrow = association (holds a reference).

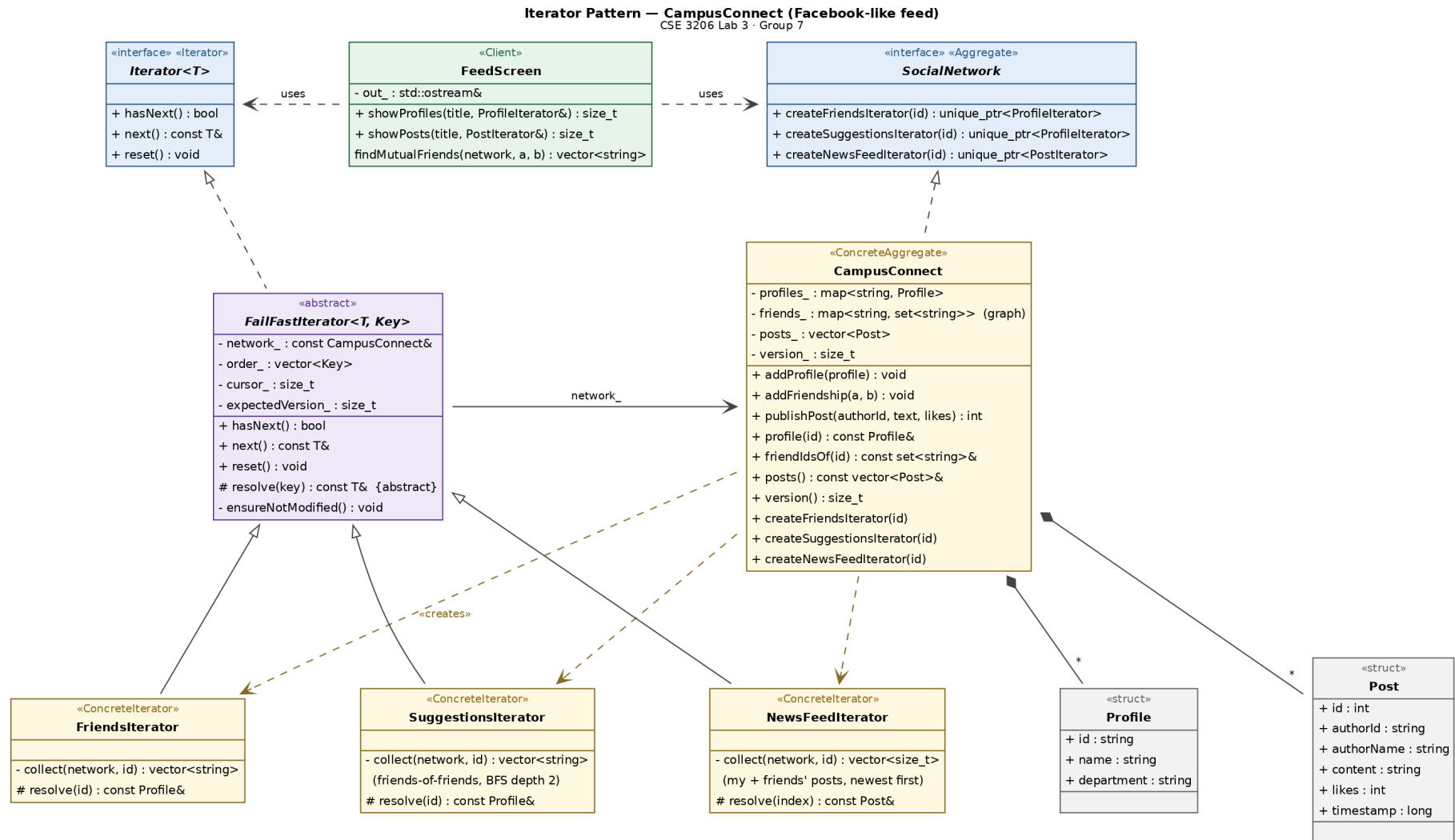


Figure 2: UML class diagram of the Iterator pattern in CampusConnect (source: docs/uml/source/iterator_class.dot).

A.7 Class Responsibilities

Class	Kind	Responsibility
<code>Iterator<T></code>	interface (template)	Declares the traversal contract <code>hasNext()</code> , <code>next()</code> , <code>reset()</code> . Documents the exceptions: <code>std::out_of_range</code> when exhausted, <code>ConcurrentModificationError</code> when the collection changed.
<code>SocialNetwork</code>	interface	Aggregate role. Declares the factory methods that hand out iterators. Clients depend on this abstraction, not on <code>CampusConnect</code> .
<code>CampusConnect</code>	concrete class	Owns profiles (map), the friendship graph (adjacency list) and posts (vector); validates input; increments <code>version_</code> on every change; creates the concrete iterators.
<code>FailFastIterator<T, Key></code>	abstract base (template)	Reusable cursor over a pre-computed order of keys. Implements <code>hasNext/next/reset</code> once, performs the fail-fast check; subclasses only supply the order and <code>resolve()</code> .
<code>FriendsIterator</code>	concrete iterator	Traversal: direct friends of a profile, alphabetical by id.
<code>SuggestionsIterator</code>	concrete iterator	Traversal: friends-of-friends (BFS depth 2) excluding self and existing friends, ranked by number of mutual friends.
<code>NewsFeedIterator</code>	concrete iterator	Traversal: posts by the profile and its friends, newest first.
<code>FeedScreen</code>	client	Renders any <code>ProfileIterator</code> or <code>PostIterator</code> with one <code>while (hasNext())</code> loop. Writes to an injected <code>std::ostream</code> so it is unit-testable.
<code>findMutualFriends()</code>	client function	Uses two independent iterators at the same time (nested loop + <code>reset()</code>).
<code>Profile, Post</code>	data (struct)	Plain value objects stored inside the network.
<code>ConcurrentModificationError</code>	exception	Signals that the network changed while an iterator was in use.

A.8 Code Implementation (step by step)

All code is standard **C++17** with no external libraries. Snippets below are copied from the repository; the file and starting line are shown above each block.

Step 1 – Define the Iterator interface

iterator/include/Iterator.hpp (from line 32)

```
template <typename T>
class Iterator {
public:
    virtual ~Iterator() = default;

    /// @return true if next() can be called at least once more.
    virtual bool hasNext() const = 0;

    /// Returns the current element and moves the cursor forward.
    /// @throws std::out_of_range if no element is left.
    /// @throws ConcurrentModificationError if the collection changed.
    virtual const T& next() = 0;

    /// Moves the cursor back to the first element (traverse again).
    virtual void reset() = 0;

protected: // polymorphic base: no slicing copies from outside (C++ Core Guidelines C.67)
    Iterator() = default;
    Iterator(const Iterator&) = default;
    Iterator& operator=(const Iterator&) = default;
    Iterator(Iterator&&) = default;
    Iterator& operator=(Iterator&&) = default;
};
```

- A **template** lets the same interface serve profiles (`Iterator<Profile>`) and posts (`Iterator<Post>`).
- `next()` returns `const T&`: no copies, and the client cannot modify the collection through the iterator.
- Copy/move operations are **protected** (C++ Core Guidelines C.67) so a derived iterator can never be *sliced* by copying it through a base reference.

Step 2 – Define the Aggregate interface (factory methods)

iterator/include/SocialNetwork.hpp (from line 22)

```
class SocialNetwork {
public:
    virtual ~SocialNetwork() = default;

    /// Friends of @p profileId in alphabetical order of their id.
    [[nodiscard]] virtual std::unique_ptr<ProfileIterator>
    createFriendsIterator(const std::string& profileId) const = 0;

    /// "People You May Know": friends-of-friends, most mutual friends first.
    [[nodiscard]] virtual std::unique_ptr<ProfileIterator>
    createSuggestionsIterator(const std::string& profileId) const = 0;

    /// News feed: posts by the user and their friends, newest first.
    [[nodiscard]] virtual std::unique_ptr<PostIterator>
    createNewsFeedIterator(const std::string& profileId) const = 0;

protected: // polymorphic base: no slicing copies from outside (C.67)
    SocialNetwork() = default;
    SocialNetwork(const SocialNetwork&) = default;
    SocialNetwork& operator=(const SocialNetwork&) = default;
    SocialNetwork(SocialNetwork&&) = default;
    SocialNetwork& operator=(SocialNetwork&&) = default;
};
```

- Each `create...Iterator()` is a **Factory Method**: the collection decides which concrete iterator to build.
- Returning `std::unique_ptr` makes ownership explicit (RAII – no manual `delete`), and `[[nodiscard]]` warns if a caller throws the iterator away.

Step 3 – Implement the Concrete Aggregate

iterator/src/CampusConnect.cpp (from line 41)

```
int CampusConnect::publishPost(const std::string& authorId,
                              const std::string& content, int likes) {
    const Profile& author = profile(authorId); // throws if unknown
    if (likes < 0) {
        throw std::invalid_argument("Likes cannot be negative");
    }
    Post post;
    post.id = nextPostId_++;
    post.authorId = author.id;
    post.authorName = author.name;
    post.content = content;
    post.likes = likes;
    post.timestamp = ++clock_;
    posts_.push_back(post);
    ++version_;
    return post.id;
}
```

iterator/src/CampusConnect.cpp (from line 96)

```
std::unique_ptr<PostIterator>
CampusConnect::createNewsFeedIterator(const std::string& profileId) const {
    return std::make_unique<NewsFeedIterator>(*this, profileId);
}
```

Every modifying method (`addProfile`, `addFriendship`, `publishPost`) increments `version_`. The factory simply passes `*this` to the new iterator – the iterator is the only object that knows the traversal order.

Step 4 – Write the traversal machinery once: FailFastIterator

iterator/include/FeedIterators.hpp (from line 31)

```
template <typename T, typename Key>
class FailFastIterator : public Iterator<T> {
public:
    bool hasNext() const override { return cursor_ < order_.size(); }

    const T& next() override {
        ensureNotModified();
        if (!hasNext()) {
            throw std::out_of_range("Iterator exhausted: no more elements");
        }
        return resolve(order_[cursor_++]);
    }

    void reset() override {
        ensureNotModified();
        cursor_ = 0;
    }

    /// Number of elements this traversal will visit in total.
    std::size_t size() const noexcept { return order_.size(); }

protected:
    FailFastIterator(const CampusConnect& network, std::vector<Key> order)
```

```

        : network_(network),
          order_(std::move(order)),
          expectedVersion_(network.version()) {}

    /// Converts a stored key into the element handed to the client.
    virtual const T& resolve(const Key& key) const = 0;

    const CampusConnect& network() const noexcept { return network_; }

private:
    void ensureNotModified() const {
        if (network_.version() != expectedVersion_) {
            throw ConcurrentModificationError(
                "CampusConnect was modified while it was being iterated");
        }
    }

    const CampusConnect& network_;
    std::vector<Key>      order_;
    std::size_t          cursor_ = 0;
    std::size_t          expectedVersion_;
};

```

- The base class stores the **order** (a vector of keys), the **cursor** and the **version** the network had when the iterator was created.
- `next()` first checks that the version is unchanged (fail-fast), then checks bounds, then returns the element and advances the cursor.
- Sub-classes only implement `resolve()` (key → element) and a static `collect()` that computes the order. This removes duplicated code from the three concrete iterators (DRY).

Step 5 – Concrete iterators = traversal algorithms

iterator/src/FeedIterators.cpp (from line 36)

```

std::vector<std::string>
SuggestionsIterator::collect(const CampusConnect& network,
                             const std::string& profileId) {
    const std::set<std::string>& myFriends = network.friendIdsOf(profileId);

    // Breadth-first search, depth 2: count how many of my friends know X.
    std::map<std::string, int> mutualCount;
    for (const std::string& friendId : myFriends) {
        for (const std::string& candidate : network.friendIdsOf(friendId)) {
            const bool isMe      = candidate == profileId;
            const bool isMyFriend = myFriends.count(candidate) > 0;
            if (!isMe && !isMyFriend) {
                ++mutualCount[candidate];
            }
        }
    }

    std::vector<std::string> order;
    order.reserve(mutualCount.size());
    for (const auto& entry : mutualCount) {
        order.push_back(entry.first);
    }
    std::stable_sort(order.begin(), order.end(),
                    [&mutualCount](const std::string& a, const std::string& b) {
                        return mutualCount.at(a) > mutualCount.at(b);
                    });
    return order;
}

```

SuggestionsIterator performs a depth-2 breadth-first search: for every friend it visits that friend's friends, skips the user and existing friends, and counts mutual friends. A stable sort ranks the most-connected suggestions first.

iterator/src/FeedIterators.cpp (from line 75)

```
std::vector<std::size_t> NewsFeedIterator::collect(const CampusConnect& network,
                                                    const std::string& profileId) {
    std::set<std::string> circle = network.friendIdsOf(profileId);
    circle.insert(profileId); // my own posts appear in my feed too

    const std::vector<Post>& posts = network.posts();
    std::vector<std::size_t> order;
    for (std::size_t i = 0; i < posts.size(); ++i) {
        if (circle.count(posts[i].authorId) > 0) {
            order.push_back(i);
        }
    }
    std::sort(order.begin(), order.end(), [&posts](std::size_t a, std::size_t b) {
        return posts[a].timestamp > posts[b].timestamp; // newest first
    });
    return order;
}
```

NewsFeedIterator stores only **indices** into the post vector (cheap), keeps posts whose author is in the user's circle, and orders them newest first.

Step 6 – The Client: one loop for every collection

iterator/src/FeedScreen.cpp (from line 25)

```
std::size_t FeedScreen::showPosts(const std::string& title, PostIterator& posts) {
    out_ << title << '\n';
    std::size_t count = 0;
    while (posts.hasNext()) { // same loop for ANY collection
        const Post& post = posts.next();
        out_ << " [t=" << post.timestamp << "] " << post.authorName << ": \""
            << post.content << "\" (" << post.likes << " likes)\n";
        ++count;
    }
    if (count == 0) {
        out_ << " (no posts)\n";
    }
    return count;
}
```

iterator/src/FeedScreen.cpp (from line 40)

```
std::vector<std::string> findMutualFriends(const SocialNetwork& network,
                                          const std::string& firstId,
                                          const std::string& secondId) {
    auto outer = network.createFriendsIterator(firstId);
    auto inner = network.createFriendsIterator(secondId); // 2nd, independent cursor

    std::vector<std::string> mutual;
    while (outer->hasNext()) {
        const Profile& candidate = outer->next();
        inner->reset(); // restart the inner traversal
        while (inner->hasNext()) {
            if (inner->next().id == candidate.id) {
                mutual.push_back(candidate.name);
                break;
            }
        }
    }
    return mutual;
}
```

`showProfiles()` and `showPosts()` contain the *same* loop. `findMutualFriends()` shows a key benefit of the pattern: two iterators over the same kind of collection run independently, and `reset()` restarts the inner one.

Step 7 – Wiring it together in main()

iterator/main.cpp (from line 57)

```
CampusConnect network; // <<ConcreteAggregate>>
buildSampleNetwork(network);
FeedScreen screen(std::cout); // <<Client>> - knows only Iterator<T>

// 1-3. One client, three different traversal algorithms -----
banner("1. FriendsIterator: Kafi's friends");
auto friends = network.createFriendsIterator("kafi");
screen.showProfiles("Friends of Kafi:", *friends);

banner("2. SuggestionsIterator: People You May Know");
auto suggestions = network.createSuggestionsIterator("kafi");
screen.showProfiles("Suggested for Kafi (most mutual friends first):", *suggestions);

banner("3. NewsFeedIterator: Kafi's news feed");
auto feed = network.createNewsFeedIterator("kafi");
screen.showPosts("Newest first, only Kafi + friends:", *feed);
```

A.9 Execution Flow (Object Interaction)

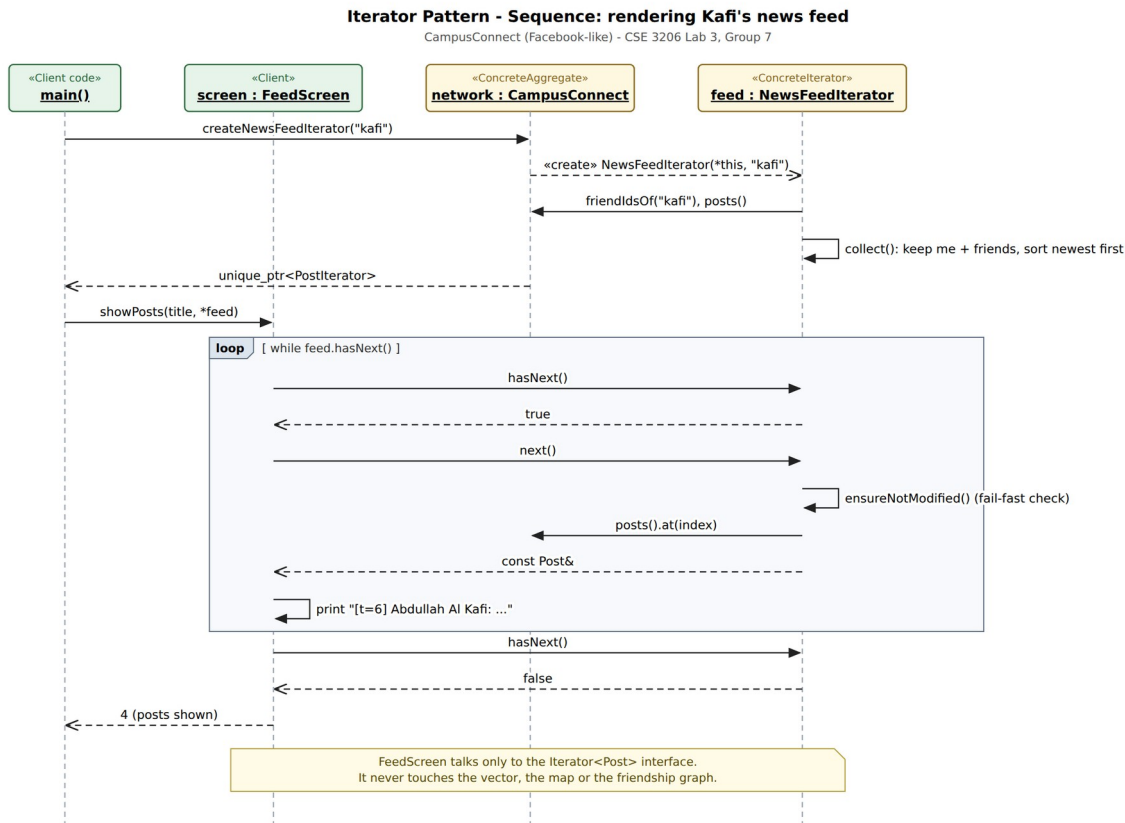


Figure 3: Sequence diagram – rendering Kafi's news feed through a NewsFeedIterator.

1. `main()` asks the **aggregate** for an iterator: `network.createNewsFeedIterator("kafi")`.
2. `CampusConnect` constructs a `NewsFeedIterator`, passing itself (`*this`).
3. The iterator's `collect()` reads Kafi's friend ids and the post list **once**, keeps posts by Kafi and his friends, sorts them newest first and remembers the network's `version()`.
4. `main()` passes the iterator to `FeedScreen::showPosts()` as a plain `PostIterator&` – the screen never learns the concrete type.
5. Loop: `hasNext()` → `true`; `next()` verifies the version (fail-fast), resolves the stored index to a `const Post&`, advances the cursor; the screen prints the post.
6. When `hasNext()` returns `false` the loop ends and the screen returns the number of posts shown (4).
7. If a post is published while an iterator is alive, the next call to `next()` throws `ConcurrentModificationError` (demo step 6); the client simply asks for a fresh iterator.

A.10 Advantages

- **Single Responsibility Principle** – storing (`CampusConnect`), traversing (iterators) and displaying (`FeedScreen`) are separate classes.
- **Open/Closed Principle** – a new ordering (e.g. `TopPostsIterator`) is a new class plus one factory method; `FeedScreen` does not change.
- **Encapsulation** – the map, graph and vector stay private; only iterators are exposed.
- **Uniform, polymorphic traversal** – one loop handles friends, suggestions and the news feed.
- **Multiple simultaneous traversals** – every iterator has its own cursor (used by `findMutualFriends`).
- **Safety** – fail-fast detection turns a silent memory bug into a clear, catchable exception.
- **Testability** – each traversal algorithm is tested in isolation (16 unit tests).

A.11 Limitations

- **Overkill for simple cases** – for one plain `std::vector`, a range-based `for` loop is simpler.
- **More classes and indirection** – one interface, one base and three iterators; a virtual call per element.
- **Snapshot cost** – our iterators compute their order when created ($O(n \log n)$ time and $O(n)$ memory for keys). A very large feed would need a *lazy* or *paged* iterator.
- **Fail-fast is detection, not prevention** – after a modification the client must request a new iterator. The design is single-threaded and not thread-safe.
- **Lifetime rule** – an iterator holds a reference to its network and must not outlive it.
- **Not STL-compatible out of the box** – our interface does not plug into range-based `for` or `<algorithm>` without a small `begin()/end()` adapter.

A.12 Real-life Applications

Application	What is iterated
Social-media feeds with infinite scrolling	posts, loaded page by page with a cursor
Music / video playlists	songs in sequential, shuffle or repeat order
File explorers and backup tools	files and folders of a directory tree
Databases	rows of a query result through a cursor
Search engines and online shops	results page by page
TV remote “channel up / down”	channels, without knowing how they are stored
Games	entities / components in a scene each frame

A.13 Industry Examples

- **C++ Standard Library** – every container provides iterators; range-based `for` uses `begin()/end()`; `std::filesystem::directory_iterator` walks folders.
- **Java** – `java.util.Iterator / Iterable` and the enhanced `for` loop. Java's *fail-fast* collection iterators throw `ConcurrentModificationException`, which inspired our `ConcurrentModificationError`.
- **C# / .NET** – `IEnumerable<T> / IEnumerator<T>`, `foreach` and `yield return`; LINQ is built on top of them.
- **Python** – the iterator protocol (`__iter__`, `__next__`) and generators.
- **JavaScript** – `Symbol.iterator`, `for...of` and generator functions.
- **JDBC** – `ResultSet.next()` is a database cursor, i.e. an iterator over query rows.
- **Web APIs with cursor-based pagination** – e.g. the Facebook Graph API (`before / after` cursors), Slack (`next_cursor`) and GitHub's GraphQL API (`endCursor / after`).

A.14 Demonstration (Live Execution)

Build and run: `build.bat run` on Windows (MinGW g++) or `./build.sh run` on Linux, macOS and Git Bash; the executable is `bin/iterator_demo`. CMake instructions are in Appendix A.

Demo step	What it proves
1. FriendsIterator	a graph traversal behind the common interface
2. SuggestionsIterator	a different algorithm (BFS depth 2 + ranking) with the same client code

Demo step	What it proves
3. NewsFeedIterator	filtering + ordering hidden inside the iterator
4. Two iterators at once	independent cursors over the same collection (mutual friends)
5. reset()	the same iterator object can traverse again
6. Fail-fast	modification during iteration is detected (<code>ConcurrentModificationError</code>)
7. Exception handling	unknown profile → <code>invalid_argument</code> ; reading past the end → <code>out_of_range</code>

Actual console output of `iterator_demo`

```
CampusConnect - Iterator Pattern Demo (CSE 3206, Group 7)

==== 1. FriendsIterator: Kafi's friends ====
Friends of Kafi:
  1. Sadaf Rahman (@sadaf, CSE)
  2. Sakila Akter (@sakila, CSE)

==== 2. SuggestionsIterator: People You May Know ====
Suggested for Kafi (most mutual friends first):
  1. Rafi Hasan (@rafi, EEE)
  2. Nusrat Jahan (@nusrat, ME)

==== 3. NewsFeedIterator: Kafi's news feed ====
Newest first, only Kafi + friends:
  [t=6] Abdullah Al Kafi: "Live demo tomorrow in the CSE 3206 lab. Wish us luck!" (40 likes)
  [t=5] Sakila Akter: "UML diagrams done, pushing to GitHub tonight" (17 likes)
  [t=3] Abdullah Al Kafi: "Iterator pattern = one loop for every collection" (25 likes)
  [t=1] Sadaf Rahman: "Lab 3 group formed: we got Iterator + Mediator!" (12 likes)

==== 4. Two iterators at once: mutual friends of Kafi and Rafi ====
  - Sadaf Rahman
  - Sakila Akter

==== 5. reset(): walk Kafi's friends again with the SAME iterator ====
Friends of Kafi (second pass):
  1. Sadaf Rahman (@sadaf, CSE)
  2. Sakila Akter (@sakila, CSE)

==== 6. Fail-fast: a new post arrives while scrolling ====
First post on screen: "Live demo tomorrow in the CSE 3206 lab. Wish us luck!"
Caught ConcurrentModificationError: CampusConnect was modified while it was being iterated
After refresh, top post: "Breaking: the demo actually works!"

==== 7. Exception handling ====
Caught invalid_argument: Unknown profile id: zara
Caught out_of_range: Iterator exhausted: no more elements

Demo finished successfully.
```

A.15 Conclusion

The Iterator pattern separates **what** a collection stores from **how** it is traversed. In CampusConnect, three very different traversals – a graph neighbourhood, a ranked breadth-first search and a filtered, time-ordered feed – are exposed through a single `Iterator<T>` interface. The UI became shorter, the containers stayed private, new orderings can be added without touching existing code, and the fail-fast check turns a dangerous memory bug into a clear exception. The same idea powers every modern language's `for-each` loop and every paginated social-media feed.

Part B — Mediator Pattern (CampusConnect Messenger)

Scenario in one line

CampusConnect Messenger is a **Messenger-like** chat service with direct messages, group chats, blocking, offline delivery and a help-desk bot. With the Mediator pattern, users never talk to each other directly. Every message goes through one **MessengerServer**, which owns all the rules.

B.1 Pattern Name

Mediator Pattern (sometimes called *Intermediary* or *Controller*).

In one sentence: *objects stop referring to each other and instead communicate through one mediator object that encapsulates the rules of their interaction.*

B.2 Category

Creational	Structural	Behavioral
—	—	✓ Mediator belongs here

Mediator is an **object-behavioral** pattern: it defines how a group of objects cooperate by moving the *interaction logic* out of the objects and into a separate mediator object.

B.3 Intent

Intent (Gang of Four, 1994)

“Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.”

Why the pattern was created:

- To replace a chaotic **many-to-many** web of references with a simple **one-to-many** star around a hub.
- To keep complex interaction rules in **one place** instead of scattering them across many classes.
- To make the individual components (**colleagues**) reusable, because they no longer depend on each other.

B.4 Problem Statement

CampusConnect Messenger must support the following behaviour:

Feature	Rule
Direct message	delivered immediately if the recipient is online
Offline delivery	if the recipient is offline, queue the message and deliver it (in order) when they come back online
Group chat (#Group-7)	deliver to every member except the sender ; only members may post
Blocking	if the recipient blocked the sender, drop the message and report Blocked
Help-desk bot	a new kind of participant that auto-replies to humans, but never to groups or other bots (no infinite loops)

Feature	Rule
History	every accepted message is recorded once, with a unique id
Validation	unknown users, empty text, non-members and users who have not logged in are rejected with exceptions

The design problem

How can N users (humans and bots) communicate under all these rules **without every user holding references to**
- and re-implementing the rules for - every other user?

B.5 Motivation – why the normal implementation is poor

Why Mediator? Direct coupling vs. a central mediator

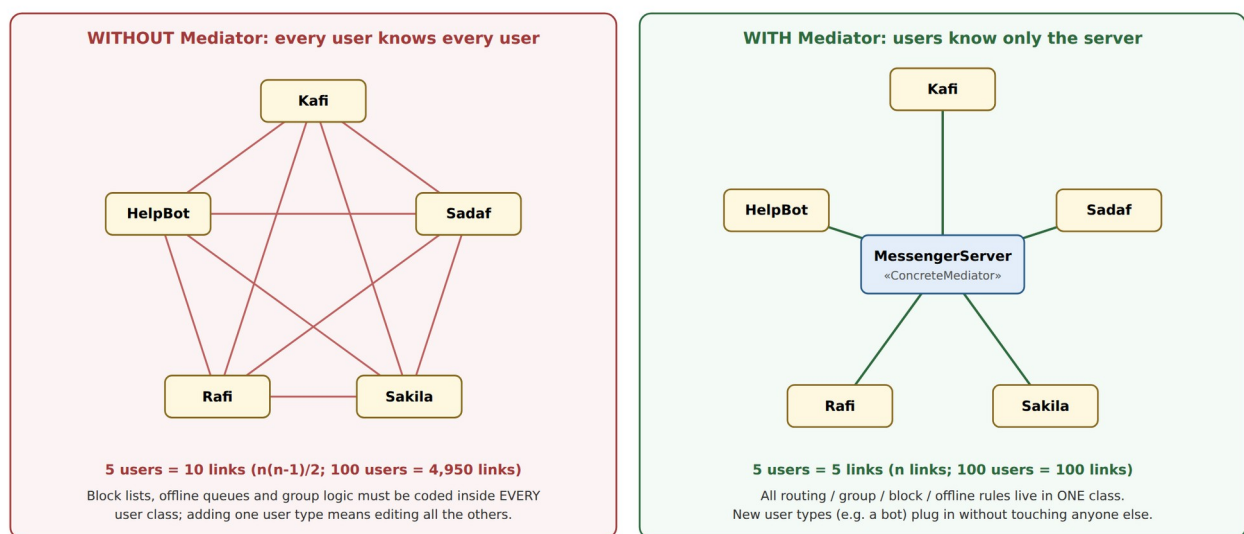


Figure 4: Without a mediator the links grow as $n(n-1)/2$; with a mediator they grow as n .

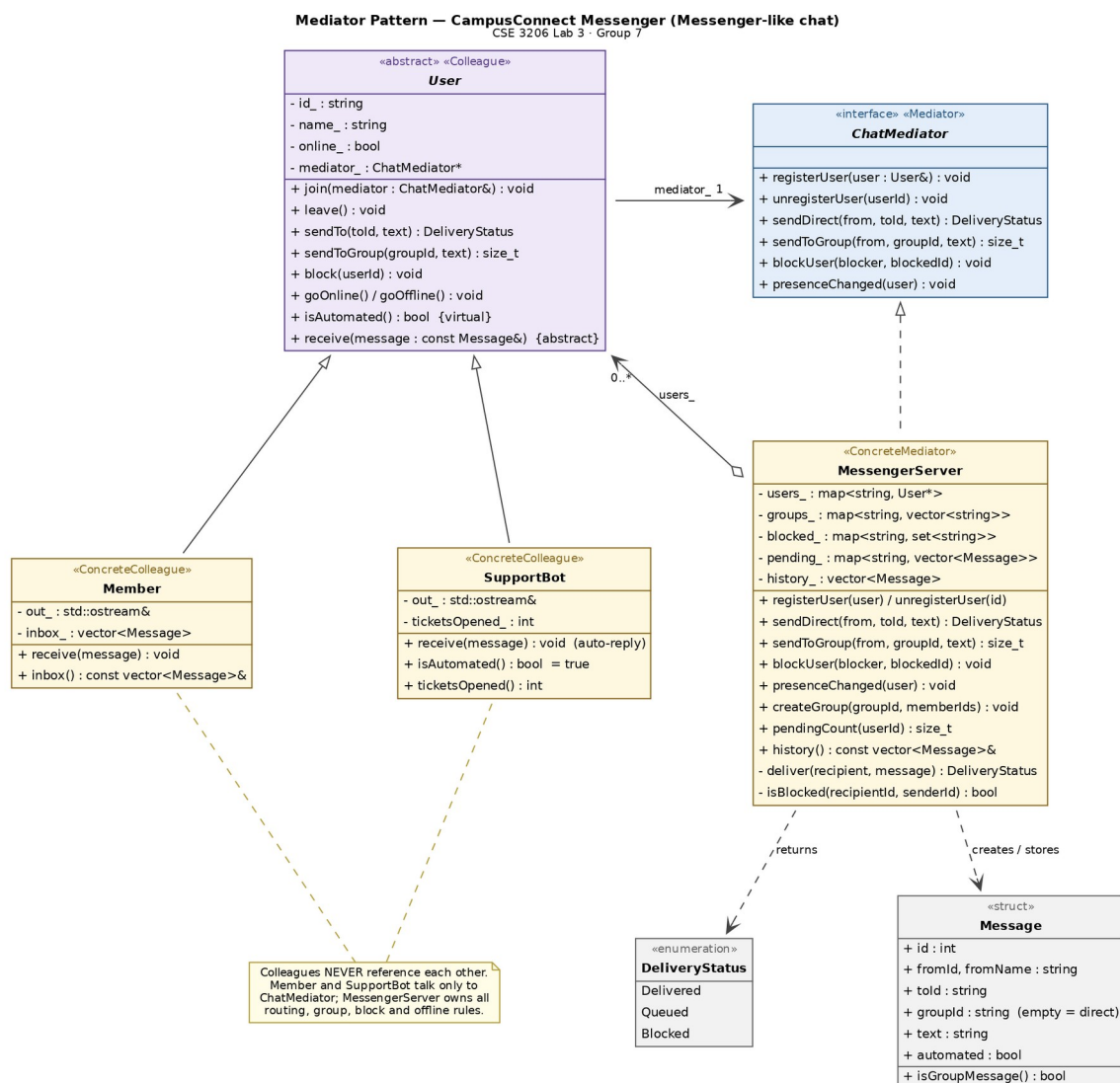
The obvious, “normal” solution is to let users call each other directly:

Naive approach (anti-example, not part of the project)

```
// NAIVE design (what we did NOT do): every user talks to every other user
class User {
public:
    void addContact(User* other) { contacts_.push_back(other); }
    void sendTo(User& to, const std::string& text) {
        if (to.hasBlocked(*this)) return; // block rule lives here
        if (!to.isOnline()) { to.queue_.push_back({text}); return; } // offline rule here
        to.receive(*this, text); // direct call to another User
    }
    void sendToGroup(std::vector<User*>& group, const std::string& text) {
        for (User* u : group) if (u != this) sendTo(*u, text); // group rule here
    }
private:
    std::vector<User*> contacts_; // N-1 pointers in EVERY user
    std::vector<Msg> queue_; // written to by OTHER users!
};
// A Bot class would have to copy all of these rules again.
```

Why this is poor:

- **Tight coupling** – every user holds pointers to other users: $n(n-1)/2$ links (5 users → 10, 100 users → 4,950).
- **Rules are scattered and duplicated** – blocking, offline queues, group fan-out and logging live inside every user class (and again in the bot).
- **Hard to change** – changing one rule (e.g. “blocked messages should be logged”) means editing many classes.
- **Hard to extend** – adding a new participant type forces the other classes to learn about it.
- **Broken encapsulation** – users write directly into each other's private queues.
- **Hard to test** – a user cannot be tested without constructing all the users it talks to.

B.6 Pattern Structure (UML Class Diagram)**Figure 5:** UML class diagram of the Mediator pattern in CampusConnect Messenger (source: docs/uml/source/mediator_class.dot).**Mapping of the classic (GoF) participants to our classes:**

GoF participant	Our class(es)	File
Mediator	ChatMediator	mediator/include/ChatMediator.hpp

GoF participant	Our class(es)	File
ConcreteMediator	MessengerServer	mediator/include/MessengerServer.hpp, src/MessengerServer.cpp
Colleague	User (abstract)	mediator/include/User.hpp, src/User.cpp
ConcreteColleague	Member, SupportBot	mediator/include/Member.hpp, SupportBot.hpp (+ .cpp)
Client	main() - creates the server and users and joins them	mediator/main.cpp
(supporting)	Message, DeliveryStatus, PermissionDenied	mediator/include/Message.hpp

The most important thing to notice in the diagram: there is **no association** between `Member` and `SupportBot`, or between any two users. The only links are *Colleague* → *Mediator* (`mediator_`) and *ConcreteMediator* → *Colleagues* (`users_`).

B.7 Class Responsibilities

Class	Kind	Responsibility
ChatMediator	interface	The only thing a colleague knows about the system: <code>registerUser</code> , <code>unregisterUser</code> , <code>sendDirect</code> , <code>sendToGroup</code> , <code>blockUser</code> , <code>presenceChanged</code> .
MessengerServer	concrete mediator	Keeps the user registry (non-owning), groups, block lists, offline queues and history. Implements every rule: routing, group fan-out, blocking, queueing and flushing, validation. Throws on invalid requests.
User	abstract colleague	Identity (<code>id</code> , <code>name</code>), online flag and a pointer to its mediator. <code>join()</code> / <code>leave()</code> (RAII: the destructor leaves automatically). Every action is delegated to the mediator; <code>receive()</code> is abstract. Non-copyable and non-movable because the server stores its address.
Member	concrete colleague	A human user. <code>receive()</code> prints a “phone notification” to an injected <code>std::ostream</code> and stores the message in its inbox.
SupportBot	concrete colleague	An automated account (<code>isAutomated() == true</code>). <code>receive()</code> opens a ticket and replies through the mediator ; ignores group messages and automated messages to prevent bot-to-bot loops.
Message	data (struct)	<code>id</code> , sender <code>id/name</code> , recipient <code>id</code> , group <code>id</code> (empty = direct), text, automated flag.
DeliveryStatus	enum	<code>Delivered</code> , <code>Queued</code> , <code>Blocked</code> – the result of a direct message.
PermissionDenied	exception	Thrown when a user posts in a group they are not a member of.

B.8 Code Implementation (step by step)

Step 1 – Define the data that travels between colleagues

mediator/include/Message.hpp (from line 13)

```
/// What happened to a direct message.
enum class DeliveryStatus : std::uint8_t {
    Delivered,    ///< recipient was online and received it immediately
    Queued,       ///< recipient offline - stored until they come online
    Blocked       ///< recipient has blocked the sender - dropped silently
};
```

mediator/include/Message.hpp (from line 31)

```
struct Message {
    int            id = 0;
    std::string fromId;
    std::string fromName;
    std::string toId;
    std::string groupId;    ///< empty for a direct (one-to-one) message
    std::string text;
    bool          automated = false;    ///< true if a bot sent it

    bool isGroupMessage() const noexcept { return !groupId.empty(); }
};
```

Step 2 – Define the Mediator interface

mediator/include/ChatMediator.hpp (from line 19)

```
class ChatMediator {
public:
    virtual ~ChatMediator() = default;

    // ---- membership -----
    virtual void registerUser(User& user) = 0;
    virtual void unregisterUser(const std::string& userId) noexcept = 0;

    // ---- interactions requested by colleagues -----
    virtual DeliveryStatus sendDirect(const User& from, const std::string& toId,
                                     const std::string& text) = 0;

    /// @return number of group members the message reached (now or queued).
    virtual std::size_t sendToGroup(const User& from, const std::string& groupId,
                                    const std::string& text) = 0;

    virtual void blockUser(const User& blocker, const std::string& blockedId) = 0;

    /// A colleague tells the mediator it went online/offline.
    virtual void presenceChanged(const User& user) = 0;
```

`User` is only forward-declared: the interface talks about users without depending on their implementation. This interface is the **only** type that colleagues depend on (Dependency Inversion Principle).

Step 3 – The Colleague base class only delegates

mediator/src/User.cpp (from line 22)

```
void User::join(ChatMediator& mediator) {
    if (mediator_ != nullptr) {
        throw std::logic_error(name_ + " is already connected to a server");
    }
    mediator.registerUser(*this);    // may throw (e.g. duplicate id)
    mediator_ = &mediator;           // only set once registration succeeded
}
```

```

void User::leave() noexcept {
    if (mediator_ != nullptr) {
        mediator_>unregisterUser(id_);
        mediator_ = nullptr;
    }
}

DeliveryStatus User::sendTo(const std::string& toId, const std::string& text) {
    return mediator().sendDirect(*this, toId, text);
}

std::size_t User::sendToGroup(const std::string& groupId, const std::string& text) {
    return mediator().sendToGroup(*this, groupId, text);
}

void User::block(const std::string& userId) { mediator().blockUser(*this, userId); }

void User::goOnline() { setPresence(true); }

void User::goOffline() { setPresence(false); }

void User::setPresence(bool online) {
    online_ = online;
    if (mediator_ != nullptr) {
        mediator_>presenceChanged(*this); // server may flush queued messages
    }
}

```

- `join()` registers with the server first and stores the pointer **only if registration succeeded** (strong exception guarantee).
- `sendTo`, `sendToGroup`, `block` contain **no business rules** – they forward `*this` to the mediator.
- `goOffline()` / `goOnline()` notify the mediator, which may flush queued messages.
- The destructor calls `leave()`, so a destroyed user can never be reached through a dangling pointer.

Step 4 – Concrete colleagues

mediator/src/Member.cpp (from line 15)

```

void Member::receive(const Message& message) {
    out_ << "    [" << name() << "'s phone] ";
    if (message.isGroupMessage()) {
        out_ << '#' << message.groupId << " | ";
    }
    out_ << message.fromName << ": " << message.text << '\n';
    inbox_.push_back(message);
}

```

mediator/src/SupportBot.cpp (from line 15)

```

void SupportBot::receive(const Message& message) {
    if (message.automated || message.isGroupMessage()) {
        return; // never answer bots or group chatter -> no infinite loops
    }
    const int ticket = nextTicket_++;
    ++ticketsOpened_;
    out_ << "    [" << name() << "]" << "opened ticket #" << ticket << " for "
        << message.fromName << '\n';

    // The reply goes back THROUGH the mediator, never directly to the sender.
    sendTo(message.fromId, "Hi " + message.fromName + ", ticket #" +
        std::to_string(ticket) +
        " created. A moderator will reply soon.");
}

```

The bot is a different kind of colleague, yet it plugged into the system **without any change to `Member` or `MessengerServer`**. Its reply goes back through the mediator exactly like a human's message.

Step 5 – The Concrete Mediator owns every rule

mediator/src/MessengerServer.cpp (from line 36)

```
DeliveryStatus MessengerServer::sendDirect(const User& from, const std::string& toId,
                                           const std::string& text) {
    requireRegistered(from);
    User& recipient = requireUser(toId);
    if (text.empty()) {
        throw std::invalid_argument("Cannot send an empty message");
    }
    if (isBlocked(toId, from.id())) {
        return DeliveryStatus::Blocked;
    }
    const Message message = makeMessage(from, toId, "", text);
    history_.push_back(message);
    return deliver(recipient, message);
}
```

mediator/src/MessengerServer.cpp (from line 51)

```
std::size_t MessengerServer::sendToGroup(const User& from, const std::string& groupId,
                                          const std::string& text) {
    requireRegistered(from);
    const auto group = groups_.find(groupId);
    if (group == groups_.end()) {
        throw std::invalid_argument("Unknown group: " + groupId);
    }
    // Copy: a receive() callback must not be able to invalidate our loop.
    const std::vector<std::string> members = group->second;
    if (std::find(members.begin(), members.end(), from.id()) == members.end()) {
        throw PermissionDenied(from.name() + " is not a member of #" + groupId);
    }
    if (text.empty()) {
        throw std::invalid_argument("Cannot send an empty message");
    }

    const Message record = makeMessage(from, groupId, groupId, text);
    history_.push_back(record);

    std::size_t reached = 0;
    for (const std::string& memberId : members) {
        const auto member = users_.find(memberId);
        const bool isSender = memberId == from.id();
        const bool isOnServer = member != users_.end();
        if (isSender || !isOnServer || isBlocked(memberId, from.id())) {
            continue;
        }
        Message copy = record;
        copy.toId = memberId;
        deliver(*member->second, copy);
        ++reached;
    }
    return reached;
}
```

mediator/src/MessengerServer.cpp (from line 95)

```
void MessengerServer::presenceChanged(const User& user) {
    requireRegistered(user);
    if (!user.isOnline()) {
        return;
    }
    const auto queue = pending_.find(user.id());
    if (queue == pending_.end()) {
        return;
    }
    // Move the queue out first so deliveries can safely enqueue new messages.
    const std::vector<Message> waiting = std::move(queue->second);
    pending_.erase(queue);
    User& recipient = *users_.at(user.id());
    for (const Message& message : waiting) {
        recipient.receive(message);
    }
}
```

mediator/src/MessengerServer.cpp (from line 164)

```
DeliveryStatus MessengerServer::deliver(User& recipient, const Message& message) {
    if (recipient.isOnline()) {
        recipient.receive(message);
        return DeliveryStatus::Delivered;
    }
    pending_[recipient.id()].push_back(message);
    return DeliveryStatus::Queued;
}
```

- Validation first (registered sender, existing recipient/group, membership, non-empty text), then the block rule, then delivery.
- `deliver()` is the single place that decides *deliver now vs queue for later*.
- Re-entrancy is handled: the member list and the offline queue are **copied/moved out before looping**, because a colleague's `receive()` (e.g. the bot) may call back into the server.

Step 6 – The Client wires everything together

mediator/main.cpp (from line 34)

```
// The server is declared FIRST so it outlives every user (users
// unregister themselves in their destructor).
MessengerServer server("CampusConnect Messenger"); // <<ConcreteMediator>>

Member      kafi("kafi", "Kafi");           // <<ConcreteColleague>>
Member      sadaf("sadaf", "Sadaf");
Member      sakila("sakila", "Sakila");
Member      rafi("rafi", "Rafi");
SupportBot  helpDesk("helpdesk", "RUET HelpDesk Bot"); // another colleague type

banner("1. Everyone joins the server (users know ONLY the server)");
kafi.join(server);
sadaf.join(server);
sakila.join(server);
rafi.join(server);
helpDesk.join(server);
std::cout << " " << server.userCount() << " accounts registered on "
          << server.serverName() << '\n';

banner("2. Direct message: both users online");
report("Kafi -> Sadaf", kafi.sendTo("sadaf", "Did you finish the Mediator UML?"));
report("Sadaf -> Kafi", sadaf.sendTo("kafi", "Yes! Pushing it to GitHub now."));
```

The server is declared before the users, so it outlives them (C++ destroys local objects in reverse order). Each user only calls `join(server)`; no user is ever given a reference to another user.

B.9 Execution Flow (Object Interaction)

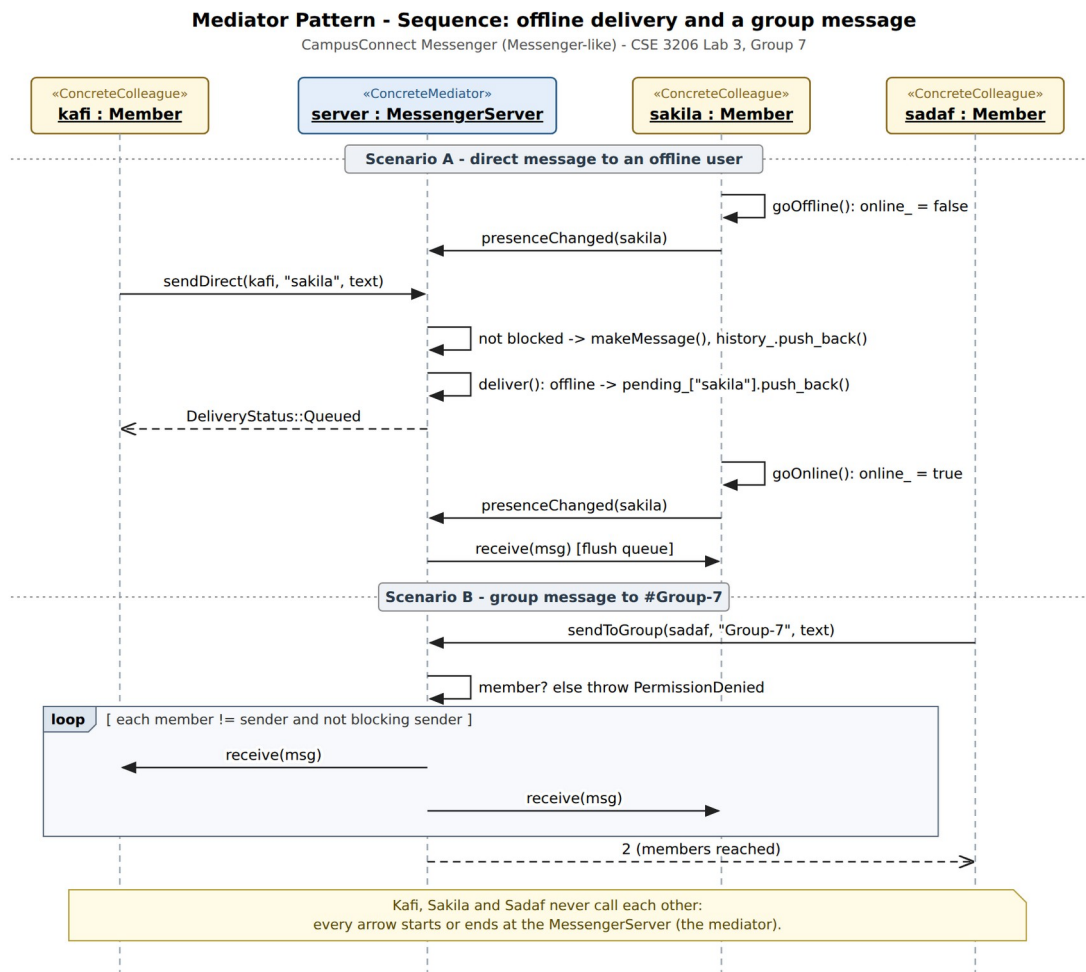


Figure 6: Sequence diagram – offline delivery (scenario A) and a group message (scenario B).

Scenario A – direct message to an offline user

1. Sakila goes offline: `sakila.goOffline()` sets `online_ = false` and calls `server.presenceChanged(sakila)`.
2. Kafi calls `kafi.sendDirect("sakila", text)`, which becomes `server.sendDirect(kafi, "sakila", text)`.
3. The server validates, checks the block list, creates the `Message` and stores it in `history_`.
4. `deliver()` sees that Sakila is offline and appends the message to `pending_["sakila"]`; Kafi receives `DeliveryStatus::Queued`.
5. Sakila comes online; `presenceChanged()` moves her queue out and calls `sakila.receive(msg)` for each waiting message.

Scenario B – group message

1. Sadaf calls `sadaf.sendToGroup("Group-7", text) → server.sendToGroup(sadaf, "Group-7", text)`.
2. The server checks that the group exists and that Sadaf is a member (otherwise `PermissionDenied`).
3. For every member except the sender who has not blocked the sender, the server calls `deliver() → kafi.receive(msg)` and `sakila.receive(msg)`.

4. The server returns 2 (members reached). Kafi, Sakila and Sadaf never called each other.

B.10 Advantages

- **Loose coupling** – each colleague knows only `ChatMediator`; links grow as n instead of $n(n-1)/2$.
- **Single Responsibility** – all interaction rules live in one class; users only represent people/bots.
- **Open/Closed for colleagues** – `SupportBot` was added without modifying `Member` or any other user.
- **Dependency Inversion** – colleagues depend on the `ChatMediator` abstraction, so another mediator (a test server, a classroom server) can be swapped in.
- **Central control point** – history/audit logging, spam filtering or rate limiting can be added in one place.
- **Reusability and testability** – users are reusable with any mediator; 17 unit tests exercise every rule through the server.

B.11 Limitations

- **Risk of a “God object”** – as rules accumulate, the mediator can become large and complex. Mitigation: split it into collaborators such as a `GroupManager` or `DeliveryPolicy` (Strategy pattern).
- **Single point of failure / bottleneck** – in a real distributed messenger the server must be replicated and scaled.
- **Indirection** – reading `User` alone does not tell you who receives a message; you must read the mediator.
- **Lifetime management** – our server stores non-owning raw pointers, so it must outlive its users. We enforce this by declaration order and `RAII leave()`; `std::weak_ptr` would be an alternative.
- **Synchronous and single-threaded** – a production messenger needs asynchronous queues, persistence and thread safety.
- **Re-entrancy must be considered** – a colleague reacting inside `receive()` calls back into the mediator (we copy lists before looping to stay safe).

B.12 Real-life Applications

Application	Who is the mediator?
Chat and messaging apps	the chat server routes messages between clients
Air-traffic control	the control tower; aircraft talk to the tower, not to each other
GUI dialog boxes / forms	the dialog enables, disables and updates widgets when another widget changes
Smart-home hub	the hub coordinates sensors, lights, locks and alarms
Ride-sharing dispatch	the dispatcher matches riders and drivers
Stock exchange	the matching engine pairs buy and sell orders
Online multiplayer games	the game server relays and validates player actions

B.13 Industry Examples

- **MediatR (.NET)** – a widely used library that implements an in-process mediator: request and notification objects are sent to the mediator, which dispatches them to handlers.
- **MVC controllers / Spring `DispatcherServlet`** – a central component receives requests and routes them to handlers, so views and models do not reference each other directly (mediator-style).

- **Redux store and front-end event buses** – components communicate through a central store/bus instead of referencing each other (mediator-style).
- **Android shared `ViewModel`** – fragments communicate through a shared ViewModel rather than calling each other directly.
- **Message brokers (RabbitMQ, Apache Kafka)** – producers and consumers are decoupled by a broker (mediator at the architecture level).
- **Messaging back-ends (XMPP servers, chat servers)** – clients send to the server, which routes to recipients, exactly like our `MessengerServer`.

B.14 Demonstration (Live Execution)

Build and run: `build.bat run` on Windows (MinGW g++) or `./build.sh run` on Linux, macOS and Git Bash; the executable is `bin/mediator_demo`. CMake instructions are in Appendix A.

Demo step	What it proves
1. Join	users register with the server and never receive references to each other
2. Direct message	routing through the mediator (<code>Delivered</code>)
3. Offline delivery	queue while offline, automatic flush on <code>goOnline()</code>
4. Group chat	fan-out to all members except the sender
5. Blocking	a central rule enforced by the server (<code>Blocked</code>)
6. SupportBot	a different colleague type replying through the mediator
7. Exception handling	<code>PermissionDenied</code> , unknown user (<code>invalid_argument</code>), not logged in (<code>logic_error</code>)
8. Statistics	history (6 messages), inbox size, tickets opened

Actual console output of mediator_demo

```
CampusConnect Messenger - Mediator Pattern Demo (CSE 3206, Group 7)

==== 1. Everyone joins the server (users know ONLY the server) ====
  5 accounts registered on CampusConnect Messenger

==== 2. Direct message: both users online ====
  [Sadaf's phone] Kafi: Did you finish the Mediator UML?
  Kafi -> Sadaf -> Delivered
  [Kafi's phone] Sadaf: Yes! Pushing it to GitHub now.
  Sadaf -> Kafi -> Delivered

==== 3. Offline delivery: the server queues the message ====
  Kafi -> Sakila -> Queued (recipient offline)
  Messages waiting for Sakila: 1
  Sakila comes online...
  [Sakila's phone] Kafi: Presentation practice at 8 PM?
  Messages waiting for Sakila: 0

==== 4. Group chat #Group-7 (sender is not echoed) ====
  [Kafi's phone] #Group-7 | Sadaf: Code review report is ready, please check.
  [Sakila's phone] #Group-7 | Sadaf: Code review report is ready, please check.
  Sadaf's group message reached 2 members

==== 5. Blocking: rule enforced centrally by the server ====
  Rafi -> Sakila -> Blocked by recipient

==== 6. A different colleague type: SupportBot replies via the server ====
  [RUET HelpDesk Bot] opened ticket #1001 for Kafi
```

```
[Kafi's phone] RUET HelpDesk Bot: Hi Kafi, ticket #1001 created. A moderator will reply soon.
Kafi -> HelpDesk -> Delivered

==== 7. Exception handling: invalid requests are rejected ====
Caught PermissionDenied: Rafi is not a member of #Group-7
Caught invalid_argument: Unknown user: zara
Caught logic_error: Guest must join a server before chatting

==== 8. Server statistics ====
Messages stored in history : 6
Kafi's inbox                : 3 messages
Tickets opened by the bot   : 1

Demo finished successfully.
```

B.15 Conclusion

The Mediator pattern turns a tangled web of user-to-user references into a clean star around `MessengerServer`. Direct messages, group chats, blocking, offline delivery and the help-desk bot all work, yet `Member` and `SupportBot` contain no routing logic and do not know each other. Every rule can be changed in one class, new kinds of participants plug in without touching existing ones, and the whole behaviour is verified by unit tests. The price is a central class that must be kept well-structured, which we document as a known limitation.

Part C — Comparing Iterator and Mediator

C.1 Side-by-side Comparison

Aspect	Iterator	Mediator
Category	Behavioral	Behavioral
Core question	How do I traverse a collection without knowing its structure?	How do many objects interact without referring to each other?
What it decouples	client ↔ the collection's internal representation	colleague ↔ colleague
Shape of the relationship	one collection, many independent cursors	many-to-many turned into a star around a hub
Where the logic lives	traversal algorithm inside iterator classes	interaction rules inside the mediator
Key participants	Iterator, ConcreteIterator, Aggregate, ConcreteAggregate, Client	Mediator, ConcreteMediator, Colleague, ConcreteColleague
SOLID principles supported	SRP, OCP	SRP, OCP, DIP
Main risk	overkill for simple lists; iterator lifetime/invalidation	mediator grows into a “God object”
In our project	friends, People You May Know, news feed	direct and group messages, blocking, offline delivery, bot

C.2 How the Two Patterns Work Together

Although our modules are independent, the patterns combine naturally in a real social platform:

- The messenger could offer `createHistoryIterator(userId)`, so a chat screen scrolls a conversation with the **Iterator** pattern while messages are still routed by the **Mediator**.
- A “Message all my friends” feature would take a `FriendsIterator` from the feed and hand each friend id to `MessengerServer::sendDirect()` – the two modules cooperate while staying decoupled.
- Inside `MessengerServer::sendToGroup()` the mediator itself *iterates* over group members; an iterator over members would hide how groups are stored.

C.3 Related Patterns

Pattern	Relationship
Factory Method	our <code>create...Iterator()</code> methods are factory methods that choose the concrete iterator
Composite	iterators are often used to walk composite (tree) structures
Memento	can capture an iterator's position so a traversal can be resumed later
Visitor	Visitor applies an <i>operation</i> to each element; Iterator only provides <i>access</i> to the elements
Observer	Observer distributes one-to-many notifications; Mediator centralises many-to-many coordination (a mediator is often implemented with Observer)
Facade	Facade gives a simpler interface to a subsystem and is one-directional; in Mediator the colleagues know and call the mediator (two-directional)

Part D — Testing and Code Quality

Correctness and code quality were checked with automated tools. Full details, including the checklist-based review, are in the separate **Code Review Report**.

Check	Tool / command	Result
Unit tests – Iterator	<code>test_iterator</code> (16 tests)	16 / 16 passed
Unit tests – Mediator	<code>test_mediator</code> (17 tests)	17 / 17 passed
Compiler warnings	<code>g++ 13</code> and <code>clang++ 18</code> , <code>-Wall -Wextra -Wpedantic -Wshadow -Wconversion</code>	0 warnings
Static analysis	<code>clang-tidy 18</code> (6 check families)	6 findings fixed, 7 accepted with reason
Memory errors / UB	<code>-fsanitize=address</code> , undefined on tests and demos	0 errors
Memory leaks	Valgrind memcheck on both demos and both test programs	0 leaks (“All heap blocks were freed”)
Test coverage	gcov line coverage of the module .cpp files	Iterator 100 %, Mediator 98.9 % *
Continuous integration	GitHub Actions: CMake build + CTest + demos	runs on every push / pull request

The two Mediator lines gcov reports as unexecuted are compiler-generated (the deleting-destructor variant of the `abstractUser` class and a closing brace after return-value optimisation); every hand-written statement is executed by the tests.

What the tests cover. Iterator: order of each traversal, empty collections, mutual friendship, ranking of suggestions, exclusion of self, newest-first feed, `reset()`, reading past the end, independent iterators, fail-fast on modification, invalid ids, input validation and the client code (including empty collections). Mediator: direct delivery, offline queue and ordered flush, blocking, invalid requests, group fan-out (sender excluded), block list and offline queue inside groups, non-member rejection, group validation, bot reply through the mediator, no bot loops, join rules, rejection of users registered on another server, automatic unregistration on destruction, input validation and history recording.

Appendices

Appendix A – How to Build and Run

Build commands

```
git clone https://github.com/Kafi2611/CSE3206-Group7-Iterator-Mediator.git
cd CSE3206-Group7-Iterator-Mediator

build.bat run          (Windows: builds into bin\, runs 33 tests, then both demos)
./build.sh run         (Linux / macOS / Git Bash: same steps)

cmake -S . -B build && cmake --build build          (any IDE / platform with CMake)
ctest --test-dir build --output-on-failure
```

Appendix B – Repository Structure

Folder layout

```
CSE3206-Group7-Iterator-Mediator/
|-- README.md                project overview + quick start
|-- GITHUB_GUIDE.md         step-by-step Git/GitHub guide for the team
|-- CMakeLists.txt          root build file (modules are optional)
|-- build.bat / build.sh    one-click build + test scripts
|-- .github/workflows/ci.yml GitHub Actions: build + test on every push
|-- iterator/               Part A - CampusConnect Feed (Iterator)
|   |-- include/            Iterator.hpp Models.hpp SocialNetwork.hpp CampusConnect.hpp
|   |                      FeedIterators.hpp FeedScreen.hpp
|   |-- src/               CampusConnect.cpp FeedIterators.cpp FeedScreen.cpp
|   |-- main.cpp            iterator demo
|-- mediator/               Part B - CampusConnect Messenger (Mediator)
|   |-- include/            Message.hpp ChatMediator.hpp User.hpp Member.hpp
|   |                      SupportBot.hpp MessengerServer.hpp
|   |-- src/               User.cpp Member.cpp SupportBot.cpp MessengerServer.cpp
|   |-- main.cpp            mediator demo
|-- tests/                  TestFramework.hpp test_iterator.cpp test_mediator.cpp
|-- docs/
|   |-- uml/                class, sequence and motivation diagrams (PNG + SVG + sources)
|   |-- Group7_Lab3_Documentation.pdf
|   |-- Group7_Code_Review_Report.pdf
```

Appendix C – References

1. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 – chapters “Iterator” and “Mediator”.
2. A. Shvets, *Refactoring.Guru – Design Patterns: Iterator and Mediator*, <https://refactoring.guru/design-patterns>
3. B. Stroustrup, H. Sutter (eds.), *C++ Core Guidelines*, <https://isocpp.github.io/CppCoreGuidelines>
4. cppreference.com – Iterator library and containers, <https://en.cppreference.com>
5. Department of CSE, RUET – CSE 3206 Lab 3: *Design Pattern Analysis, Implementation and Code Review* (lab handout).